

CRYPTODY: Cryptographic Misuse Analysis of IoT Firmware via Data-flow Reasoning

Jianing Wang
jianing@comp.nus.edu.sg
National University of Singapore

Shanqing Guo*
guoshanqing@sdu.edu.cn
Shandong University

Wenrui Diao
diaowenrui@link.cuhk.edu.hk
Shandong University

Yue Liu
230209311@seu.edu.cn
Southeast University
QI-ANXIN Group

Haixin Duan
duanhx@tsinghua.edu.cn
Tsinghua University
Zhongguancun Laboratory
Quancheng Laboratory

Yichen Liu
liuyic@iu.edu
Indiana University Bloomington

Zhenkai Liang*
liangzk@comp.nus.edu.sg
National University of Singapore

ABSTRACT

Cryptographic techniques form the foundation of the security and privacy of computing solutions. However, if cryptographic APIs are not invoked correctly, they can result in significant security problems. In this paper, we abstract the intricate crypto misuse detection problem as a data-flow reasoning task. Towards this end, we propose CRYPTODY, a novel logic-inference-based framework for detecting crypto misuses via reasoning about data flows on multi-architecture IoT firmware images. It carries out cross-architecture analysis, with detection strategies to reduce false positives and false negatives, such as cross-flow misuse inference. To evaluate the effectiveness of CRYPTODY, we conducted a large-scale experiment on 1,431 firmware images from 16 vendors. Our evaluation shows that 46% of the firmware images have high-risk misuses and 95% have at least one cryptographic misuse. In total, we find 6,624 potential crypto misuses, with 760 being cross-flow misuses that are not detected by existing solutions. We have responsibly disclosed portions of our findings to the relevant vendors. From the feedback, we note that CRYPTODY has a low false-positive rate for the confirmed misuses. Some typical cases have been assigned CVEs and fixed by the vendors.

CCS CONCEPTS

• Security and privacy → Malware and its mitigation.

KEYWORDS

Cryptographic misuse, IoT firmware, Data-flow analysis

*Corresponding authors

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
RAID 2024, September 30–October 02, 2024, Padua, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0959-3/24/09
<https://doi.org/10.1145/3678890.3678914>

ACM Reference Format:

Jianing Wang, Shanqing Guo, Wenrui Diao, Yue Liu, Haixin Duan, Yichen Liu, and Zhenkai Liang. 2024. CRYPTODY: Cryptographic Misuse Analysis of IoT Firmware via Data-flow Reasoning. In *The 27th International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2024)*, September 30–October 02, 2024, Padua, Italy. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3678890.3678914>

1 INTRODUCTION

The security of widely-deployed Internet of Things (IoT) devices has become an increasing concern. In these Internet-facing devices, cryptographic techniques form the foundation of security and privacy solutions, including safeguarding network communication and protecting device data. However, improper cryptographic API usage (*crypto misuse* in short) can lead to severe security vulnerabilities. Common examples of crypto misuse include using exposed secret keys, employing ECB mode in symmetric ciphers, or utilizing insufficient-size keys in asymmetric ciphers. Such misuses often occur due to factors such as complex API design [1, 35], inexperienced developers [33], and copying flawed code snippets from popular technology websites [20].

Recent studies [16, 28, 31, 40, 42, 48, 55] reveal that crypto misuse is a common problem across various platforms. For instance, 88% of Android applications and 85% of Apache projects suffer from crypto misuses [16, 42]. Efforts to detect crypto misuse automatically fall into two categories: static analysis [16, 28, 42, 55] and dynamic analysis [40, 48], with their own pros and cons. Dynamic tools monitor the execution of the target program, offering more accurate detection results. However, they face the code coverage problem. In contrast, static tools scrutinize either the source code or the binary of the target program. Specifically, they detect crypto misuse by checking the data flows of crypto APIs based on predefined detection patterns. While static tools offer extensive code coverage, they often suffer from accuracy issues, leading to false positives.

Crypto misuse is also prevalent in IoT devices, as they are commonly deployed in resource-constrained environments where security might be sacrificed to meet performance requirements. However, crypto misuse detection for IoT devices has not been well studied yet, with only one study available [55]. On IoT platforms,

dynamic analysis is notably challenging to implement due to the difficulty of firmware rehosting [32]. Hence, we focus on static analysis in this paper.

Static analysis tools for IoT devices face the following challenges. Firstly, detecting misuses involving multiple data flows poses challenges due to the intricate semantics and dependencies of crypto APIs. We list two typical cases of such misuses that involve multiple data flows (*cross-flow* misuse for short): i) A single crypto algorithm can involve various APIs in common IoT crypto libraries. For instance, the misuse of using low exponent in RSA encryption can involve two different data flows between three APIs in OpenSSL. ii) Multiple correct data flows that intersect with each other may result in misuse. For example, consider the scenario where a securely-generated RSA key is used both for encryption and signing [25]. Individually, each data flow is correct, but their combination is problematic. Unfortunately, existing solutions lack such cross-flow misuse detection capability. Secondly, ignoring APIs' contextual information can lead to false positives. For example, while hash functions like SHA1 and MD5 are outdated, they are still acceptable in specific contexts, such as HMAC scenarios. Nevertheless, existing solutions flag all SHA1 and MD5 usage as misuse [42]. Thirdly, the detection patterns and implementations must be flexible and extensible to keep pace with evolving crypto libraries and emerging crypto misuses. However, existing solutions implemented by imperative languages offer limited adaptability and extensibility.

The gist of detecting crypto misuse relies heavily on reasoning about the intricate dependencies of data flows using predefined detection patterns. Notably, logic inference has demonstrated its strength in such reasoning tasks, evident from its applications in domains like binary disassembly [21] and memory forensics [41]. In these tasks, given a set of *facts* (data), the inference engine deduces new facts by repeatedly applying the logic *rules* until no new facts can be deduced. To shed light on this approach, we abstract the intricate misuse detection problem as a data-flow reasoning task: data flows serve as facts, forming the search space for the logic inference engine; meanwhile, detection patterns are translated into concise and expressive logic rules to capture the dependencies and context information, steering the inference towards pinpointing crypto misuses. In this process, single-flow misuses are first deduced and then act as known facts, aiding in inferring cross-flow misuses. Moreover, the inherent flexibility and extensibility of logic inference facilitates the seamless addition of new rules in a declarative manner, thus sidestepping the limitations of traditionally handcrafted imperative implementations.

This paper presents CRYPTODY, the first logic inference-based system for detecting crypto misuses via reasoning about data flows of IoT firmware images. At a high level, CRYPTODY takes multi-architecture binary files of firmware images as input and lifts them to an intermediate representation (IR). Next, it performs data-flow tracking on the IR to build a data-flow graph (DFG), which captures the semantics and dependencies of the data flows. Finally, guided by logic rules, it infers crypto misuses based on the facts which encode the rich information present in the DFG and pre-defined crypto API models. CRYPTODY employs a set of optimization strategies to reduce false positives and false negatives, such as cross-flow misuse inference (Sec 3.3.4), and cross-file misuse inference (Sec 3.3.5). Moreover, it achieves high efficiencies with a series of refinement

strategies, such as on-demand lifting (Sec 3.2.2), and reusing summaries to avoid redundant analysis (Sec 3.3.5).

We evaluate the effectiveness and efficiency of CRYPTODY on large-scale real-world firmware images and our benchmark. We carry out a systematic experiment on 1,431 real-world firmware images from 16 vendors. The results reveal that 95% of these images exhibit at least one crypto misuse, while 46% have high-risk crypto misuses. Upon manually inspecting 301 of the discovered 6,624 potential crypto misuses, we note that the false positive rate is 6.0%. Notably, CRYPTODY demonstrates its advantages in detecting complex cross-flow misuses that existing solutions [42, 55] cannot handle. 760 of the discovered misuses are identified as cross-flow misuses. By considering contextual information, CRYPTODY reduces false positives by 17% for specific misuses. Moreover, to facilitate further exploration of solutions on crypto misuse detection, we build an IoT-specific benchmark named CRYPTODYBENCH, which contains 65 test cases, including intraprocedural, interprocedural, and cross-file cases. CRYPTODY passed all the test cases in the benchmark. The source code of CRYPTODY and the benchmark are available at [50].

We responsibly disclosed 1,835 misuses to 14 vendors, generating vendor-specific reports to streamline the resolution process. Among the reactions, i) 8 vendors responded: 4 acknowledged and are actively addressing some of the reported misuses, while the other 4 who are still evaluating them. ii) Two CVEs have been assigned to the reported misuses. For instance, we discovered a vulnerability of using a constant AES key in DrayTek's firmware, which affected hundreds of firmware images from 64 devices. It was rated as critical by NIST and fixed by DrayTek. iii) Vendors' feedback indicates a low false positive rate of 1.3% among the 299 confirmed misuses.

In summary, we make the following contributions:

- We present CRYPTODY, the first logic inference-based system for detecting crypto misuse via reasoning about data flows in multi-architecture IoT firmware images. Our system addresses the limitations of existing solutions, such as their inability to detect cross-flow misuse and their neglect of APIs' contextual information.
- We implement CRYPTODY and systematically evaluate it on both large-scale real-world firmware and a benchmark. The results show that CRYPTODY achieves high effectiveness and efficiency in detecting crypto misuses. Artifacts are available at [50].
- We responsibly disclosed portions of our findings to the relevant vendors. Based on their feedback, we note that CRYPTODY exhibits a low false-positive rate for the confirmed misuses. Several typical cases have been acknowledged and fixed by the vendors, and two CVEs have been assigned to the reported misuses.

2 BACKGROUND & MOTIVATION

In this section, we provide an overview of the background of crypto misuse and present our insights into formulating the crypto misuse detection problem as a data-flow reasoning task.

2.1 Types of Crypto Misuse

Crypto misuse has been extensively studied in previous research works [16, 25, 40, 42, 55] and documented in public sources [9, 23,

52]. Table 1 summarizes various types of crypto misuse, categorized based on the type of crypto algorithms involved. Each misuse comes with a concise description, a CWE [9] number associated with it, and an attack method commonly employed.

Symmetric Ciphers Misuses. Symmetric ciphers use the same cryptographic keys for both encryption of plaintext and decryption of ciphertext. Common misuses of symmetric ciphers are summarized as follows. V-01: Use constant crypto keys [16]. This allows attackers to easily obtain the key and perform arbitrary encryption or decryption operations. V-02: Use static initialization vectors (IVs) in the CBC mode [16]. This can enable attackers to perform chosen plaintext attacks (CPA) on the cryptographic system. V-03: Use the ECB mode [16]. Similar to the use of static IVs, this can lead to chosen plaintext attacks. V-04: Use insecure symmetric cipher algorithms [42], such as DES, IDEA, Blowfish, RC2, and RC4. These algorithms may have design flaws or inadequate encryption strength, making them vulnerable to attacks. V-05: Use constant passwords for password-based encryption (PBE) [10]. This practice exposes the secrets as an attacker can easily obtain them. V-06: Use constant salts for PBE [16]. Constant salts make it easier for attackers to launch brute-force attacks. V-07: Use fewer than 1,000 iterations for PBE [16]. Insufficient iterations in PBE weaken the security and make brute-force attacks more feasible.

Asymmetric Ciphers Misuses. Asymmetric ciphers, also known as public-key ciphers, employ key pairs consisting of public keys (widely disseminated) and private keys (known only to the owner). We outline common misuses of asymmetric ciphers as follows: V-08: Use keys with insufficient size [23]. For instance, key sizes of less than 2048 bits in RSA and DSA are considered insecure. In the case of elliptic curve cryptography (ECC), keys with fewer than 224 bits should not be used. V-09: Use the RSA algorithm without Optimal Asymmetric Encryption Padding (OAEP) [13]. Utilizing RSA with the OAEP scheme helps eliminate the impact of predictable common text and enhances security. V-10: Use low public exponents in RSA [52]. RSA keys generated with low public exponents, such as 3 or 17 instead of the recommended value 65537, are vulnerable to Coppersmith’s attack [52]. V-11: The misuse involves using the same RSA keys for both encryption and signing [17]. This practice is considered insecure.

Hash Function Misuses. Hash algorithms can calculate a fixed-length hash value from messages of arbitrary sizes. The hash algorithm is a one-way function and can be used to guarantee the integrity of messages. V-12: Use insecure hash algorithms, such as MD2, and MD4 [42]. V-13: Although SHA1 and MD5 are not recommended for general use, they are still acceptable when used in the context of HMAC [18]. Ignoring their usage in HMAC scenarios can lead to false positives.

PRNG Misuses. Pseudorandom number generator (PRNG) generates a sequence of numbers whose properties approximate sequences of random numbers. The PRNG-generated sequence relies on an initial value, i.e., the seed, which may contain truly random values. Common misuses of PRNG are as follows. V-14: Use weak PRNG [12]. For example, the rand function in libc is not considered cryptographically strong. V-15: Use predictable or constant PRNG seeds [11]. For example, common insecure seeds include constant values, the current time, the process ID, and so on. Both V-14 and V-15 can cause the pseudorandom number to be predictable.

Cross-flow Misuse. Crypto misuses that involves multiple data flows of APIs are challenging to detect in common IoT crypto libraries, such as OpenSSL and Libgcrypt. These misuses necessitate tracking all related data flows, identifying the dependencies between them, or capturing the contextual information, such as V-01, V-02, V-10, V-11, V-12, V-13, and V-15.

High-risk Misuse. The selection criteria of high-risk misuses are based on CryptoGuard [42], which categorizes misuses’ severity based on the attacker’s gain and the attack difficulty. For instance, misuses of using predictable secrets (V-01 and V-05) are immediately exploitable and significantly advantage attackers. For V-12, commercially available rainbow tables allow attackers to easily obtain pre-images of MD5 and SHA1 hashes for typical passwords. Hash collisions for these algorithms enable attackers to forge digital signatures or compromise message integrity. Thus, these misuses are classified as high-risk.

2.2 Motivating Examples

We present two motivating examples to emphasize the necessity and significance of cross-flow analysis. The call chains displayed in the figures are simplified for clarity.

Figure 1a illustrates an instance of V-11, where a single RSA key is used for both encryption (blue) and signing (orange). Existing solutions would not flag this as a misuse since the key is securely generated in each data flow. However, this is a blatant misuse from the perspective of cross-flow analysis.

Figure 1b depicts an example of V-13, showcasing MD5 usage in different contexts. Initially, MD5 is chosen as the digest algorithm (lines 2-9). While MD5 is insecure for cryptographic hash (line 22), it is acceptable in HMAC context (line 16). Unfortunately, existing solutions flag all MD5 usage as insecure, resulting in false positives in HMAC scenarios.

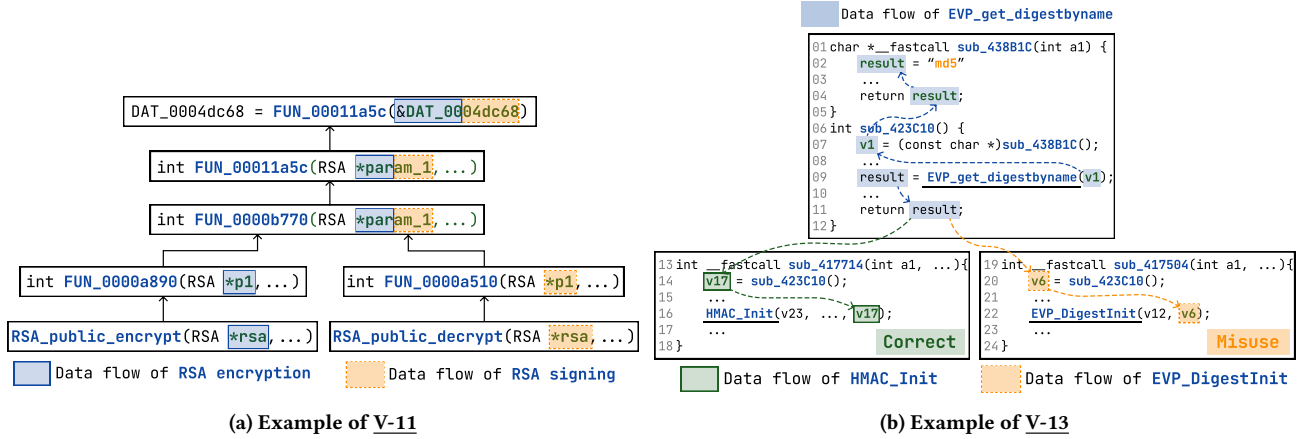
2.3 Challenges to Existing Solutions

Taking into account the aforementioned examples and the distinctive characteristics of IoT platforms, we present the following challenges of existing approaches.

- **Handling cross-flow misuses.** We empirically observe that many misuses require checking multiple data flows of crypto APIs in common IoT crypto libraries, such as OpenSSL and Libgcrypt. As such, the capability for detecting cross-flow misuses is essential for a crypto misuse detection solution. However, cross-flow analysis is not a trivial task due to the intricacies of crypto APIs, such as their rich semantics and complex dependencies. Unfortunately, existing solutions [16, 28, 31, 40, 42, 55] do not appear to detect such misuses, as they only take into account single data flows. The inability to detect cross-flow misuses also negatively impacts the accuracy of existing solutions. False positives are inevitable in these solutions since they ignore the contextual information of crypto APIs, as illustrated in the second example.
- **Efficiency in large-scale analysis.** Efficiently analyzing substantial volumes of firmware images poses performance challenges for existing solutions [55]. In practice, vendors commonly release numerous firmware images for various products. Each firmware image may contain dozens, or even hundreds, of binaries (averaging 216 in our dataset), and each binary

Table 1: Summary of crypto misuse issues. * signifies the misuse issue has a high severity.

ID	Cross-flow	Misuse Issues	Type	CWE	Attack Method
V-01 *	✓	Use constant cryptographic keys	Symmetric Ciphers	321	Exposed Secrets
V-02	✓	Use static IVs in CBC mode		329	CPA
V-03		Use ECB mode		327	CPA
V-04		Use insecure symmetric ciphers (e.g., DES, IDEA)		327	Brute-force
V-05 *		Use constant passwords for PBE		259	Exposed Secrets
V-06		Use constant salts for PBE		760	Brute-force
V-07		Fewer than 1,000 iterations for PBE		916	Brute-force
V-08		Use keys of insufficient size	Asymmetric Ciphers	326	Brute-force
V-09		Use RSA algorithm without OAEP		780	CPA
V-10	✓	Use low public exponents for RSA		311	Coppersmith
V-11	✓	Use the same key for encryption and signing		320	[25]
V-12 *		Use insecure hash algorithms (e.g., MD2, MD4)	Hash	328	Hash Collision
V-13	✓	Use insecure hash algorithms in non-HMAC scenarios (e.g., SHA1, MD5)	Hash & MAC	328	Hash Collision
V-14		Use weak pseudorandom number generator	PRNG	338	Predictability
V-15	✓	Use predictable or constant PRNG seeds		337	Predictability

**Figure 1: Motivating examples in real-world firmware.**

may consist of numerous functions. Consequently, constructing an extensive data flow graph encompassing all binaries in a firmware introduces significant computational complexity. Unfortunately, this is the approach adopted by the existing solution [55].

- **Flexibility and extensibility.** Crypto API designs can differ across various versions and libraries. This necessitates detection solutions that are flexible enough to cover various designs of APIs. However, existing solutions [16, 28, 31, 40, 42, 55] often manually design and implement complex detection algorithms, which can be error-prone and time-consuming. In addition, these solutions lack the necessary extensibility to adapt to the evolving crypto libraries and new misuses.

In summary, the effectiveness and accuracy of existing solutions are hindered by their inability to handle cross-flow misuses and their neglect of contextual information. Furthermore, their performance and scalability are impeded by the computational challenges posed by vast IoT firmware datasets and diverse crypto APIs design.

2.4 Misuse Detection as Reasoning

This line of work lies in reasoning about the complex dependencies between data flows of crypto APIs. Specifically, researchers map the empirical knowledge of crypto misuse to specific detection

patterns and then manually craft them into detection algorithms through a combination of static program analysis and heuristics. Subsequently, these algorithms are applied to inspect the data flows of crypto APIs for potential misuses.

We observe that logic inference has proven its effectiveness in solving such reasoning tasks in various domains, such as binary disassembly [21], memory forensics [41], and fusing application and audit logs [53]. In these reasoning tasks, given a set of rules and facts, the logic inference engine automatically infers new facts by repeatedly applying the rules to the facts until a fix point is reached. Thus, to shed light on this approach, we abstract the intricate misuse detection problem into a reasoning task. Leveraging the advances in logic inference, we propose a novel detection framework to overcome the aforementioned challenges. In this framework, data flows serve as facts, forming the search space for the inference engine; crypto policy rules are translated into concise and expressive logic rules that describe the dependencies as well as the contextual information of data flows, steering the inference engine towards deducing misuses. By decoupling the logic rules from the detection implementation, our framework gains extensibility, allowing it to adapt to evolving crypto libraries and emerging crypto misuses. Moreover, this framework is specifically tailored for IoT firmware, employing analysis summaries to avoid redundant analysis and reduce false negatives.

3 APPROACH

3.1 Overview

Figure 2 illustrates an overview of CRYPTODY. At a high level, it takes firmware images as input and detects crypto misuses by reasoning about the data flows. CRYPTODY consists of two key components: 1) a *data-flow graph builder*, responsible for performing data-flow tracking on an IR lifted from multi-architecture binary files, and 2) a *misuse reasoner*, which infers crypto misuses across multiple data flows on the DFG.

The key insight driving our system is to *formulate the intricate crypto misuse detection problem as a data-flow reasoning task*. Full details will be given in Section 3.2 and Section 3.3, while we outline the workflow here. Initially, our system extracts binary files from the given firmware image. Leveraging the symbols of binary files, it constructs a dependency graph to establish the analysis order and filter out irrelevant files. Subsequently, the *data-flow graph builder* lifts the binary files to Ghidra’s IR, P-Code [37], to perform data-flow tracking. The tracking is guided by predefined API models to ensure accurate and efficient tracking. Consequently, a data-flow graph is constructed, serving as the search space for the *misuse reasoner*. Finally, the reasoner automatically infers crypto misuses guided by the crypto policy rules.

In addition, it classifies the discovered misuses to identify similar ones, typically found in shared components and representing high-risk vulnerabilities. Furthermore, we offer automatically generated reports to assist vendors in addressing the identified misuses.

3.2 Data-Flow Graph Builder

The DFG builder (*the builder* for short) encodes information present in binary files into a data-flow graph which serves as a search space for further reasoning tasks.

3.2.1 Preprocessing. We undertake two preprocessing steps before initiating data-flow tracking, including binary extraction and API model construction.

Binary Extraction. Binary files must be extracted from firmware images ❶. These binaries are then analyzed to obtain their symbols, strings, and library dependencies. Note that stripped binaries are not a concern, as we only require the symbols for dynamic libraries; debugging information is not needed.

API Model Construction. To guide data-flow tracking and misuse reasoning, we construct 329 API models covering 7 crypto libraries. Each model consists of a function signature, a parameter of interest, the corresponding library, the type of misuse, and constraints on the parameter if applicable. In the DFG construction phase, the builder utilizes these models to filter out irrelevant binary files and specify the sources of data-flow tracking. In the misuse reasoning phase, the reasoner leverages these models to infer crypto misuses. We present additional information of API models in Appendix A.

3.2.2 Constructing Dependency Graph. From the extracted data, we create a dependency graph (DG) to represent inter-binary dependencies. DG is crucial for on-demand lifting and guiding the order of analysis. Firstly, since decompiling binary files is quite time-consuming, we do not need to perform static analysis on all binary files. Instead, we can obtain a crypto-related subgraph by recursively searching all the predecessors of standard crypto

libraries in the whole DG. Then the builder will lift them to IR ❷. Secondly, DG guides the order of data-flow tracking analysis. In practice, it is common for vendors to use wrapper libraries of standard crypto libraries to provide a more convenient interface. As such, the capability for handling cross-file data-flow tracking is crucial for IoT misuse detectors. The existing solution [55] addresses such cases by building a large cross-file graph. However, this approach is highly inefficient as it involves redundant analysis of shared components across different firmware images and leads to a significant memory overhead. Instead, we devise a more efficient approach leveraging DG to address this challenge, which will be discussed in Section 3.3.5.

3.2.3 Tracking Data Flows. The builder constructs a DFG by tracking the data flows of crypto APIs defined in our API models. In particular, a tracking process starts from the parameters or return values of the target function. The goal is to identify all the IR opcode statements that define or use the target parameter or return value. These opcode statements are referred to as *def-use sites*.

This tracking process consists of intraprocedural tracking (within a single function) and interprocedural tracking (across multiple functions). It is worth noting that the process supports backward and forward tracking, which is determined by either the semantics of the crypto API or the opcodes of P-Code IR. An example, illustrated in Figure 3, demonstrates tracking the data flow of the fourth parameter of the `AES_cbc_encrypt` function in the `libnrorouter` file, which is an IV for AES-CBC mode. The tracking process shows that two constant values (marked in orange) are used as the IV, which is a misuse of `V-02`.

3.2.4 Building Data-Flow Graphs. The final step ❸ of the builder is to build data-flow graphs. To bolster the misuse reasoner, we introduce a specialized DFG representation characterized as a directed acyclic graph. Each node in this graph represents a statement of P-Code IR, which can be a call site of a crypto API or a def-use site of a target parameter or return value. To clearly distinguish different tracked variables, specifically Varnode in P-Code, within the same statement, we also record both the tracking direction and the index of the target variable for each node. Additionally, each edge represents a data flow or a dependency between two variables in two nodes. The DFG features the following properties:

1) Data flows of different variables are integrated into a single graph to facilitate cross-flow analysis. Since our analysis leverages the static single assignment (SSA) form, tracking a Φ variable [51] can lead us along different paths, such as variables manipulated by the `MULTIEQUAL` opcode [22] in P-Code. In such instances, each path is tracked separately, thus establishing data flows from root node (target variable) to all child nodes (def-use sites).

2) To differentiate between distinct data flows, we make each node unique and distinguishable, taking into account multiple factors, such as the P-Code statement, the index of the target variable, and the tracking direction. This distinction is crucial as tracking the same variable in different directions—whether backward or forward—yields different semantics and leads to various def-use sites, even if the P-Code statements are the same. Thus, by differentiating the direction of the edge and the node, we effectively prevent any misinterpretation of the semantics of the P-Code statements.

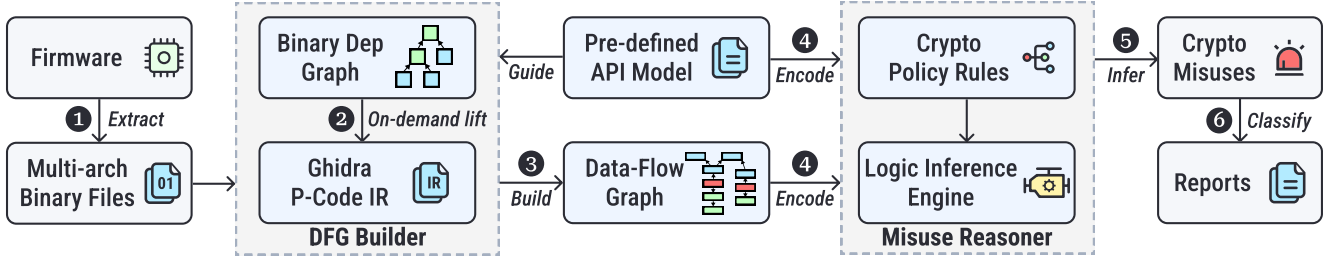


Figure 2: Overview of CRYPTODY.

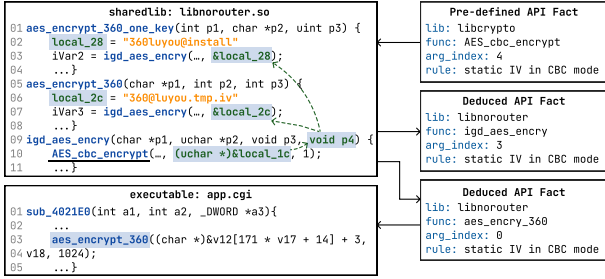


Figure 3: An example of cross-file data-flow tracking of 360 router firmware.

3.3 Misuse Reasoner

The misuse reasoner abstracts the intricate misuse detection problem as a reasoning task. The reasoning process involves: 1) Encoding DFG and crypto API models into facts. 2) Deducing intermediate facts from initial facts. 3) Inferring crypto misuses.

We choose Datalog [26] as the reasoning language, a popular logic inference language that is widely used in program analysis and verification. A Datalog program is composed of a collection of rules and facts. A *rule* is a logical inference presented in the form *head* : − *body*, wherein both the *head* and *body* are conjunctions of literals. A literal can be either a predicate or a negated predicate, while a predicate is a function symbol characterized by a fixed arity. A *fact*, on the other hand, is a predicate with all of its arguments instantiated. The Datalog program is evaluated through a repetitive application of rules to facts until a fixpoint is achieved, i.e., no new facts can be deduced from the known facts.

3.3.1 Encoding Facts. Two categories of information ④ are encoded into facts: the DFG and the API models.

As shown in the *Types* section of Figure 4, we consider two fundamental domains: strings, denoted as $\mathbb{S}$, and numbers, denoted as $\mathbb{Z}$. Further, we introduce the following sub-domains: addresses $\mathbb{A} \subseteq \mathbb{Z}$, P-Code statements $\mathbb{P} \subseteq \mathbb{S}$, opcodes $\mathbb{O} \subseteq \mathbb{S}$, the index of the tracked variable $\mathbb{I} \subseteq \mathbb{Z}$, the direction of tracking $\mathbb{D} \subseteq \{0, 1\}$, with 0 indicating backward and 1 indicating forward, string extension field $\mathbb{SE} \subseteq \mathbb{S}$, and number extension field $\mathbb{ZE} \subseteq \mathbb{Z}$. For example, the string extension stores the function name for a CALL opcode node, while the number extension can store the decimal value for a constant node. Specifically, the index of the tracked variable $\mathbb{I}$ is defined as follows: −1 signifies the output of the statement, inputs start from 0. In a call site node, 0 represents the call site itself, and

parameters begin from 1. These sub-domains can be merged to establish a more complex data type, i.e., *nodetype*, in the DFG.

As illustrated in the *Facts* section of Figure 4, the predicates declared in *DFG* represent the initial set of facts extracted from the data-flow graphs. Meanwhile, the predicates declared in *API* represent the facts derived from the API models. We note that not all API models can be straightforwardly compiled into grouped facts, due to the considerable design diversity of crypto APIs. For exceptional cases, we opt to compile them directly into a set of rules.

3.3.2 Deducing Facts. Prior to inferring crypto misuses, intermediate facts are deduced from initial facts and basic rules outlined in Figure 4. Note that the basic rules in Figure 4 are not exhaustive; we only showcase a subset of them that pertain to the following sections.

Initially, we introduce *domain constraints* to restrict the values of the domain variables, such as the opcode, parameter index, string extension, and number extension. Subsequently, we present *Node Constraints* to denote constraints of the nodes in the DFG. For example, *isCallNode*($X : \mathbb{N}$) implies that the node X is a node whose opcode is CALL in Ghidra’s IR. Next, we introduce the basic rules of the *DFG*. For instance, *reach*($X : \mathbb{N}, Y : \mathbb{N}$) indicates that a path exists from node X to node Y in the DFG. Note that *reach* is a transitive closure of the edge relation, which will recursively deduce all paths from node X to node Y . It is straightforward to see that *depend*($X : \mathbb{N}, Y : \mathbb{N}$) can be deduced from *reach*($X : \mathbb{N}, Y : \mathbb{N}$), as a path from X to Y indicates a dependency between X and Y in the DFG. Finally, we present a portion of the *API* rules. For example, *isSymKey*($X : \mathbb{N}$) implies that node X is a call site of symmetric ciphers with a key parameter.

3.3.3 Inferring Single-flow Crypto Misuses. With known and deduced facts, we progress to misuse detection—employing the crypto policy rules within the logic reference engine. For this purpose, we refer to *crypto policy rules* as a set of concise yet expressive Datalog rules.

To reason about misuses involving using constants, such as V-01, V-02, V-05, and V-06, we query the dependencies between the call sites of crypto APIs and the constant nodes in the DFG. The following example on the left demonstrates how we infer V-01.

```

1  keyShouldNotBeConst(X, Y) :-
2      isSymKey(X),
3      isConstNode(Y),
4      DFG.depend(X, Y).

```


Figure 4: Summary of Types, Facts and Basic Rules of the Misuse Reasoner**Types**

Sub-domains Address $\mathbb{A} \subseteq \mathbb{Z}$, PCode Statement $\mathbb{P} \subseteq \mathbb{S}$, Opcode $\mathbb{O} \subseteq \mathbb{S}$, Index $\mathbb{I} \subseteq \mathbb{Z}$, Direction $\mathbb{D} \subseteq \{0, 1\}$,
String Extension $\mathbb{SE} \subseteq \mathbb{S}$, Number Extension $\mathbb{ZE} \subseteq \mathbb{Z}$

Node type $\mathbb{N} = [\text{Address} : \mathbb{A}, \text{Statement} : \mathbb{P}, \text{Opcode} : \mathbb{O}, \text{Index} : \mathbb{I}, \text{Direction} : \mathbb{D}, \text{SE} : \mathbb{SE}, \text{ZE} : \mathbb{ZE}]$

Facts

DFG $\text{node}(X : \mathbb{N})$ $\text{edge}(X : \mathbb{N}, Y : \mathbb{N})$
API (V-01) $\text{symKeyAPI}(\text{Func} : \mathbb{SE}, \text{Index} : \mathbb{I})$ (V-02) $\text{symIvAPI}(\text{Func} : \mathbb{SE}, \text{Index} : \mathbb{I})$
(V-04) $\text{insecureSymFuncAPI}(\text{Func} : \mathbb{SE})$ (V-07) $\text{pbeIterAPI}(\text{Func} : \mathbb{SE}, \text{Index} : \mathbb{I})$
(V-14) $\text{insecurePRNGFuncAPI}(\text{Func} : \mathbb{SE})$...

Basic Rules

<i>Opcode Constraint</i>	$\text{opCon}(X : \mathbb{N}, O : \mathbb{O})$	$\vdash \text{node}(X), X = [A, P, O, I, D, SE, ZE]$
<i>Index Constraint</i>	$\text{indexCon}(X : \mathbb{N}, I : \mathbb{I})$	$\vdash \text{node}(X), X = [A, P, O, I, D, SE, ZE]$
<i>SE Constraint</i>	$\text{seCon}(X : \mathbb{N}, SE : \mathbb{SE})$	$\vdash \text{node}(X), X = [A, P, O, I, D, SE, ZE]$
<i>ZE Constraint</i>	$\text{zeCon}(X : \mathbb{N}, ZE : \mathbb{ZE})$	$\vdash \text{node}(X), X = [A, P, O, I, D, SE, ZE]$
<i>Node Constraints</i>	$\text{isCallNode}(X : \mathbb{N})$	$\vdash \text{node}(X), \text{opCon}(X, \text{"CALL"})$
	$\text{isFuncCallSite}(X : \mathbb{N}, F : \mathbb{SE}, I : \mathbb{I})$	$\vdash \text{isCallNode}(X), \text{seCon}(X, F), \text{indexCon}(X, I)$
	$\text{isConstNode}(X : \mathbb{N})$	$\vdash \text{node}(X), \text{opCon}(X, \text{"CONST"})$
	$\text{isNonZeroConstNode}(X : \mathbb{N})$	$\vdash \text{isConstNode}(X), \text{ze} > 0, \text{zeCon}(X, \text{ze})$
<i>DFG</i>	$\text{node}(X : \mathbb{N})$	$\vdash \text{edge}(X, _)$
	$\text{node}(X : \mathbb{N})$	$\vdash \text{edge}(_, X)$
	$\text{reach}(X : \mathbb{N}, Y : \mathbb{N})$	$\vdash \text{edge}(X, Y)$
	$\text{reach}(X : \mathbb{N}, Y : \mathbb{N})$	$\vdash \text{reach}(X, Z), \text{edge}(Z, Y)$
	$\text{depend}(X : \mathbb{N}, Y : \mathbb{N})$	$\vdash \text{reach}(X, Y)$
<i>API</i>	(V-01) $\text{isSymKey}(X : \mathbb{N})$	$\vdash \text{symKeyAPI}(F, I), \text{isFuncCallSite}(X, F, I)$
	(V-02) $\text{isSymIv}(X : \mathbb{N})$	$\vdash \text{symIvAPI}(F, I), \text{isFuncCallSite}(X, F, I)$
	(V-04) $\text{isInsecureSymFunc}(X : \mathbb{N})$	$\vdash \text{insecureSymFuncAPI}(F), \text{isFuncCallSite}(X, F, I), I \geq 0$
	...	

Certain rules require to impose constraints on their domains, such as V-07, V-08, V-09, and V-10. The following example of V-07 adds a constraint on the number extension ZE of the constant node Y, asserting that the iteration count of the PBE algorithm should exceed 1000.

```

1 pbeIterShouldNotBeLessThan1000(X, Y) :-
2   isPBEIter(X),
3   isConstNode(Y),
4   DFG.depend(X, Y),
5   Iter < 1000,
6   zeCon(Y, Iter).
```

As the design of APIs differs for certain rules, such as V-03, V-04, and V-12, we employ different strategies to identify them. The subsequent V-03 example illustrates how we detect these misuses by checking the presence of the API call sites or by imposing constraints on the values passed to the APIs. In this specific example, the value '1' being passed to the third parameter of gcry_cipher_open signifies that the padding mode is set to ECB.

```

1 isInsecureSymEcbMode(X, Y) :-
2   DFG.depend(X, Y),
3   Func = "gcry_cipher_open",
4   ParamIndex = 3,
5   isFuncCallSite(X, Func, ParamIndex),
6   isConstNode(Y),
7   Arg = 1, zeCon(Y, Arg);
8   ...
9
10 isInsecureSymEcbModeCallSite(X) :-
11   insecureSymEcbModeAPI(Func),
12   ParamIndex >= 0,
13   isFuncCallSite(X, Func, ParamIndex).
14
15 shouldNotUseECBMode(X) :-
16   isInsecureSymEcbModeCallSite(X);
17   isInsecureSymEcbMode(X, _).
```

3.3.4 Inferring Cross-flow Misuses. We employ different strategies for detecting cross-flow misuses, including V-01, V-02, V-10, V-11, V-12, V-13, and V-15. The two motivating examples described in Section 2.2 serve to illustrate our method for identifying such misuses.

Regarding the first example (V-11), we notice that distinct data flows of RSA encryption and signing depend on the same input, i.e., the same RSA key. That means a node $Z : \mathbb{N}$ exists in the data flows of both encryption and signing. We can succinctly represent the rule as follows.

```

1 RSACannotUseSameKey(X, Y, Z) :-
2   isRSAEncryptionKey(X),
3   isRSASigningKey(Y),
4   node(Z),
5   DFG.depend(X, Z),
6   DFG.depend(Y, Z).
```

To avoid false positives caused by neglecting contextual information, like the second motivating example (V-13), we introduce context rules. For instance, any usage of the MD5 or SHA1 in HMAC contexts will be excluded from the detection results.

```

1 hmacCanUseWeakHash(X, Y) :-
2   isHmac(X),
3   isMD5(Y),
4   DFG.depend(X, Y);
5   isHmac(X),
6   isSHA1(Y),
7   DFG.depend(X, Y).
8
9 shouldNotUseInsecureHashFunction(X) :-
10  isInsecureHashFuncCallSite(X),
11  !hmacCanUseWeakHash(_, X);
12  isInsecureHashFuncInt(X, _),
13  !hmacCanUseWeakHash(_, X).
```

3.3.5 Inferring Cross-file Misuses. In real-world IoT firmware, developers often use wrapper libraries of standard crypto libraries

for convenience, which can inadvertently lead to misuses spanning multiple files. Existing approaches, such as CryptoREX [55], tackle this by constructing an extensive graph encompassing all binaries. However, given that a single firmware image can have hundreds of binaries (our dataset averages 216), this approach can be incredibly resource-intensive. Hence, aiming to detect such cross-file misuses while enhancing the efficiency of large-scale analysis, we introduce a unique cross-file inference approach based on two observations:

- False negatives caused by cross-file misuses can be reduced by dynamic API fact deduction. Specifically, using the DG from Section 3.2.2, we first examine wrapper libraries that directly rely on standard crypto libraries, then evaluate the dependent files. Thus, API facts dynamically derived from the self-defined wrapper APIs form a knowledge base for subsequent analysis. For instance, as depicted in Figure 3, the executable file `app.cgi` invokes the `aes_encrypt_360` function from the `libnorouter.so` library. Since the `igd_aes_encrypt` function is a wrapper of the `AES_cbc_encrypt` function, we can deduce that the first deduced fact, which indicates that the third parameter of the `igd_aes_encrypt` function is an IV. Further, as the `aes_encrypt_360` function is a wrapper for the `igd_aes_encrypt` function, we can deduce the second fact that the `aes_encrypt_360` function has a crypto misuse of using static IV. As there is no need to track the parameters of the `aes_encrypt_360` function, we set the tracking index to 0. Without cross-file inference in the `libnorouter.so` library, the misuse in the `app.cgi` file would remain undetected.
- Reusing the summary can significantly improve the efficiency of large-scale analysis. Vendors often share base components across different products to reduce development costs. In our dataset, 46% of binary files are shared across various firmware images. Therefore, by reusing summaries, i.e., dynamically deduced API facts and inferred misuses, of these shared binaries, we can significantly reduce the analysis time.

4 IMPLEMENTATION

We implemented a prototype of CRYPTODY using 7,585 lines of Java, Python, and Datalog code. CRYPTODY integrates several off-the-shelf tools, including Ghidra [36] for binary analysis, Binwalk [29] for firmware image extraction, and Soufflé [26] for data-flow reasoning.

Enhancements for Ghidra’s Def-Use Analysis. To locate the def-use sites, the builder leverages a low-level def-use analysis interface provided by Ghidra. However, standard off-the-shelf IRs, such as Ghidra’s P-Code [37], fall short in effective data-flow tracking. The reasons are twofold. Firstly, they only offer intraprocedural def-use analysis. Consequently, we design an interprocedural tracking algorithm that is capable of recursively tracking data flows within each intraprocedural tracking phase and chain these flows together. Secondly, the scope of their intraprocedural analysis is inherently restricted. For example, the def-use sites of variables allocated on the stack or manipulated via the `INDIRECT` opcode are often overlooked by Ghidra. To mitigate these challenges, we implement a heuristic-based search algorithm that can locate the def-use sites of these variables based on their address and the semantics of P-Code opcodes.

IoT-specific Crypto Misuse Detection Benchmark. Despite the availability of numerous cryptographic misuse detection tools,

there remains an absence of a standardized metric for evaluating their detection accuracy on IoT platforms. Current benchmarks mainly focus on Java APIs (MASC [3], CryptoAPI-Bench [2], Droid-Bench [4], and Ghera [34]) and the Windows environment (Juliet test suite [38]). Given the distinct characteristics of IoT platforms, such as various CPU architectures, closed-source firmware, and unique APIs, existing benchmarks do not adequately cater to the assessment needs of crypto misuse detection tools on IoT platforms. To bridge this gap and facilitate future research, we build and release the first IoT-specific crypto misuse detection benchmark, termed CRYPTODYBENCH. It contains 65 test scenarios, covering 12 types of misuses, including symmetric and asymmetric ciphers, hash functions, and PRNGs. It provides intraprocedural, interprocedural and cross-file cases. Additionally, it provides source code along with multi-architecture binaries (x86, ARM, and MIPS), thereby facilitating its use with cross-architecture analysis tools for IoT. The detailed description of the benchmark is available in Appendix B.

Sink Identification. In our analysis process, we classify sinks into two categories: API call sites and specific definitions with constant values. API call sites are defined within the API models. For instance, the `AES_ecb_encrypt` function serves as a sink (and also a source) for the detection of V-03, whereas the `time` function acts as a sink in the analysis of the source function `srand` for identifying V-15. For the category of specific definitions, the data-flow tracking concludes upon identifying constants such as strings or integers. Identifying integer values is relatively straightforward. However, statically identifying constant strings is not trivial, as they are typically represented as pointers. To tackle this, we adopt a conservative strategy where a constant string is either a pointer to the read-only data (rodata) section or a hexadecimal value. Moreover, if the target value is subject to modification, we no longer consider it constant and halt the tracking process.

Tracking Depth. We set a threshold for the maximum number of tracking steps. Should this threshold be reached without locating sinks, the tracking process is terminated. Specifically, we set the intra-procedural tracking depth to 15 and the inter-procedural tracking depth to 7. One step is taken for each assignment or function call. The threshold is set to ensure the efficiency of the tracking process. We have empirically determined these values to be sufficient for the vast majority of cases.

Extensibility. It is crucial to underscore the modular design of CRYPTODY, which simplifies its extensibility in two key dimensions: crypto APIs and, most notably, crypto policy rules. Firstly, the decoupling inference component achieves an extensibility that exceeds existing solutions, which traditionally intertwine detection and static analysis components. Furthermore, it employs a declarative language to express logic rules, resulting in superior readability and comprehension in contrast to traditional imperative code. Secondly, it is straightforward to extend API models with JSON format, ensuring adaptability with the evolving crypto APIs.

Misuse Classification. To avoid duplicate identification of misuses in different firmware images, we classify them by the similarity of certain features ⑥. Misuses are classified by: API models, tracked variable values, and file names. We argue that checking the hash values of files is the cheapest method to identify similar misuses. However, this approach suffers from accuracy issues due to custom modifications. Compiler differences, such as different optimization options, also contribute to the inaccuracy.

5 EVALUATION

Our evaluation of CRYPTODY answers the following questions.

- Q1:** What are the security findings in real-world firmware?
- Q2:** How do IoT vendors respond to our findings?
- Q3:** How effective is CRYPTODY in inferring crypto misuses?
- Q4:** How does CRYPTODY compare to state-of-the-art tools?
- Q5:** What is the performance overhead of CRYPTODY?

Experimental Setup. CRYPTODY was deployed on a Ubuntu 20.04 server with Intel Xeon E5-2600 2.00GHz CPU and 64G RAM. Our real-world IoT firmware data set is established in cooperation with a security company. It consists of 1,431 successfully extracted firmware images from 16 vendors, such as NETGEAR, TP-Link, and ASUS. The device types include but not limited to routers, smart speakers, smart cameras, and network-attached storage (NAS).

5.1 (Q1) Findings of Real-World Firmware

Overview of the findings. As listed in Table 2, we carried out a large-scale real-world experiment consisting of 1,431 IoT firmware images from 16 vendors. In total, we discovered 6,624 potential crypto misuse bugs. 46% of the firmware images have high-risk misuses and 95% have at least one misuse.

Findings of cross-flow crypto misuses. 760 of the detected 6,624 misuses are cross-flow misuses. With the contextual information, CRYPTODY reduces false positives by 17% for specific misuses.

Detailed statistics of specific misuses. We select five representative misuses to provide detailed statistics in Figure 5, aiming to provide a clear insight into the distribution of misuses in real-world firmware. The detailed descriptions of these misuses will be discussed later in this section. For other misuses, we believe the percentage shown in Table 2 sufficiently illustrates real-world firmware misuses.

5.1.1 Statistics of Top-4 Misuses. The analysis reveals that the top four misuses occur in over 50% of the firmware images, which should be of great concern to developers. They include usage of predictable or constant PRNG seeds (V-15), insecure symmetric ciphers (V-04), insecure hash algorithms like MD5 and SHA1 in non-HMAC scenarios (V-13), and ECB mode (V-03).

Concerning V-15 in Figure 5a, 87% of the identified insecure seeds were predictable, with the remainder being constant values. Predictable seeds are often derived from sources such as the current time (e.g., time), process ID (e.g., getpid), and user ID (e.g., getuid). Regarding V-04, Figure 5b shows that DES is the most frequently misused symmetric cipher, followed by RC2, Blowfish, RC4, and IDEA. This indicates that a concerning misunderstanding regarding DES's security appears prevalent among IoT developers. Lastly, Figure 5c reveals a common preference for using AES with ECB mode in V-03. While developers recognize AES as a secure algorithm, they often overlook the inherent insecurity of ECB mode.

5.1.2 Misuse Statistics of Exposed Secrets. For V-01, 10 out of 16 vendors and 4.5% of the firmware used constant crypto keys in symmetric ciphers. For V-05, 11.9% of the firmware used constant passwords, where the most commonly found function was crypt from libc.

Suggestions: These misuses pose a significant threat as attackers can easily extract these sensitive secrets via simple reverse-engineering. To mitigate such risk, vendors can adopt a variety of techniques, such as the use of hardware security modules (HSMs), obfuscation technique, or firmware hardening.

5.1.3 Misuse Statistics of Insufficient Encryption Strength. Regarding V-06, 29.9% of the firmware used constant salts. Figure 5d reveals that many developers used only one count iteration for V-07, far below the recommended 1,000 iterations. Figure 5e shows frequent use of 1024-bit RSA keys for V-08, less than the recommended 2048 bits or more.

Suggestions: These findings expose a common practice of sacrificing security for efficiency, particularly given the limited computational resources in IoT devices. However, such negligent practices render IoT devices vulnerable to brute-force attacks. Developers should prioritize security, dedicating sufficient resources to adopt robust encryption strategies. This shift can significantly lower the likelihood of brute-force attacks, bolstering the overall security of IoT devices.

5.1.4 Misuse Statistics of CPA Attacks. Regarding V-02 (1.0%) and V-09 (10.8%), developers are encouraged to employ random IVs and secure padding schemes. Such practices can increase the unpredictability of the plaintext and thereby complicate potential attack efforts.

5.1.5 Misuse Statistics of Using Broken or Risky Crypto Algorithms. For V-12, 42.8% of firmware images are found to use outdated MD2 and MD4. It is highly recommended that developers switch to SHA-2 algorithm. Regarding V-14, common misused APIs for weak PRNGs include RAND_pseudo_bytes and BN_pseudo_rand_range. We advocate for the use of RAND_bytes in OpenSSL.

Suggestions: Developers should stay updated with crypto advancements and promptly replace outdated crypto algorithms, regarding V-03, V-04, V-12, V-13, and V-14. Using outdated algorithms may not introduce functional bugs, but it does not preclude potential exploitation either.

5.1.6 Other Misuse Statistics. Concerning to V-10 (3.3%), developers primarily use the secure public exponent 65537, but some still use low public exponents like 3, 17, and 35. For optimal balance between security and performance, 65537 is generally recommended. In our analysis, we found it alarming that the most upvoted answer on Stack Overflow [39] incorrectly suggests using 3 as a public exponent. This can lead to many developers to directly replicate this misuse in their code.

Suggestions: Developers should exercise caution when adopting code from public forums, as top-voted answers can still harbor concealed bugs. We strongly encourage developers to thoroughly check any code, rather than directly copying and pasting without verification.

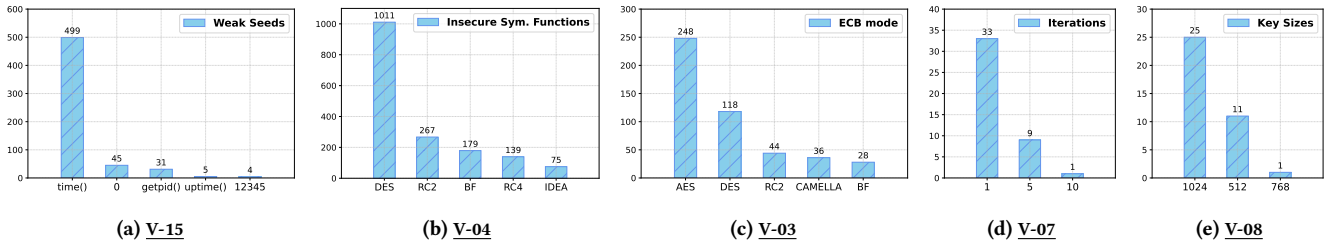
5.2 (Q2) Response from IoT Vendors

At the time of writing this paper, as shown in Table 3, we responsibly disclosed the 1,835 discovered misuses to 14 vendors, with the exception of 2 vendors who did not provide any valid email. We have received responses from 8 vendors, with 4 confirming a

Table 2: Evaluation results of crypto misuse detection experiments. FM represents the number of firmware images. FM-High represents firmware with high-risk crypto misuses. FM-One represents firmware with at least one crypto misuse.

Vendor	FM	FM-High%	FM-One%	V-01	V-02	V-03	V-04	V-05	V-06	V-07
Tenda	214	9.3%	85.0%	3 (0.5%)	0 (0.0%)	64 (15.0%)	128 (48.6%)	0 (0.0%)	2 (12.6%)	6 (6.5%)
ASUS	154	92.2%	98.1%	0 (0.0%)	2 (1.9%)	123 (89.6%)	330 (93.5%)	2 (83.8%)	4 (48.7%)	6 (25.3%)
TP-Link	143	33.6%	98.6%	7 (5.6%)	3 (4.9%)	59 (40.6%)	232 (84.6%)	0 (0.0%)	17 (15.4%)	5 (12.6%)
NETGEAR	126	65.1%	96.0%	2 (8.7%)	2 (3.2%)	68 (65.1%)	283 (84.9%)	0 (0.0%)	9 (27.0%)	4 (11.9%)
TRENDnet	119	27.7%	96.6%	1 (1.7%)	0 (0.0%)	38 (32.8%)	165 (65.5%)	6 (4.2%)	12 (13.4%)	2 (3.4%)
ipTIME	115	24.3%	100.0%	0 (0.0%)	0 (0.0%)	21 (47.0%)	48 (100.0%)	1 (0.9%)	4 (95.7%)	0 (0.0%)
Ubiquiti	113	50.4%	98.2%	6 (22.1%)	0 (0.0%)	85 (46.0%)	245 (97.3%)	0 (0.0%)	6 (23.9%)	4 (25.7%)
Dahua	93	55.9%	94.6%	2 (3.2%)	0 (0.0%)	57 (61.3%)	125 (80.6%)	0 (0.0%)	6 (14.0%)	3 (44.1%)
D-Link	75	25.3%	98.7%	0 (0.0%)	0 (0.0%)	39 (46.7%)	146 (65.3%)	1 (2.7%)	1 (1.3%)	2 (5.3%)
PELCO	61	86.9%	90.2%	10 (6.6%)	0 (0.0%)	55 (86.9%)	159 (88.5%)	1 (32.8%)	1 (39.3%)	3 (27.9%)
Yealink	58	48.3%	100.0%	1 (10.3%)	0 (0.0%)	4 (91.4%)	27 (91.4%)	0 (0.0%)	1 (65.5%)	0 (0.0%)
Synology	46	82.6%	100.0%	0 (0.0%)	2 (2.2%)	60 (71.7%)	186 (97.8%)	12 (28.3%)	23 (52.2%)	7 (17.4%)
poly	39	38.5%	100.0%	1 (7.7%)	0 (0.0%)	52 (84.6%)	134 (100.0%)	0 (0.0%)	3 (15.4%)	1 (5.1%)
ALFA	32	68.8%	100.0%	0 (0.0%)	0 (0.0%)	10 (96.9%)	17 (96.9%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
Zyxel	28	78.6%	100.0%	0 (0.0%)	0 (0.0%)	27 (85.7%)	65 (100.0%)	0 (0.0%)	3 (35.7%)	1 (17.9%)
AirLive	15	26.7%	100.0%	1 (6.7%)	0 (0.0%)	3 (40.0%)	33 (93.3%)	1 (6.7%)	1 (6.7%)	0 (0.0%)
Total	1431	46.3%	95.8%	33 (4.5%)	9 (1.0%)	516 (54.5%)	1671 (81.6%)	24 (11.9%)	84 (29.9%)	43 (13.7%)

Vendor	V-08	V-09	V-10	V-11	V-12	V-13	V-14	V-15	Total
Tenda	0 (0.0%)	9 (6.5%)	0 (0.0%)	0 (0.0%)	21 (9.3%)	74 (37.9%)	11 (7.5%)	70 (81.3%)	388
ASUS	2 (26.0%)	12 (22.1%)	0 (0.0%)	12 (53.9%)	66 (86.4%)	183 (97.4%)	19 (54.5%)	57 (98.1%)	818
TP-Link	12 (35.0%)	12 (11.2%)	0 (0.0%)	3 (0.7%)	34 (31.5%)	167 (93.7%)	20 (23.1%)	97 (93.0%)	668
NETGEAR	5 (14.3%)	1 (0.8%)	7 (5.6%)	0 (0.0%)	140 (64.3%)	229 (70.6%)	16 (23.0%)	118 (79.4%)	884
TRENDnet	1 (1.7%)	0 (0.0%)	1 (1.7%)	0 (0.0%)	47 (22.7%)	113 (73.1%)	10 (18.5%)	85 (94.1%)	481
ipTIME	2 (3.5%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	11 (23.5%)	38 (93.9%)	4 (47.8%)	14 (100.0%)	143
Ubiquiti	4 (40.7%)	8 (3.5%)	0 (0.0%)	0 (0.0%)	103 (30.1%)	110 (93.8%)	13 (47.8%)	38 (60.2%)	622
Dahua	0 (0.0%)	5 (43.0%)	0 (0.0%)	1 (1.1%)	16 (55.9%)	44 (62.4%)	10 (54.8%)	31 (91.4%)	300
D-Link	0 (0.0%)	6 (1.3%)	2 (21.3%)	0 (0.0%)	25 (22.7%)	86 (90.7%)	8 (10.7%)	51 (98.7%)	367
PELCO	2 (9.8%)	7 (67.2%)	1 (8.2%)	0 (0.0%)	26 (86.9%)	87 (88.5%)	11 (62.3%)	25 (86.9%)	388
Yealink	3 (22.4%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	7 (48.3%)	29 (100.0%)	2 (93.1%)	12 (77.6%)	86
Synology	4 (28.3%)	4 (6.5%)	0 (0.0%)	2 (17.4%)	74 (73.9%)	193 (82.6%)	10 (28.3%)	87 (91.3%)	664
poly	1 (7.7%)	0 (0.0%)	3 (17.9%)	0 (0.0%)	29 (38.5%)	89 (94.9%)	7 (30.8%)	58 (35.9%)	378
ALFA	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	7 (68.8%)	31 (93.8%)	2 (31.2%)	16 (100.0%)	83
Zyxel	4 (42.9%)	0 (0.0%)	1 (32.1%)	0 (0.0%)	24 (78.6%)	69 (89.3%)	3 (14.3%)	41 (89.3%)	238
AirLive	2 (20.0%)	0 (0.0%)	1 (6.7%)	0 (0.0%)	4 (13.3%)	36 (100.0%)	5 (73.3%)	29 (100.0%)	116
Total	37 (14.7%)	64 (10.8%)	9 (3.3%)	18 (6.5%)	428 (42.8%)	1255 (79.5%)	118 (34.5%)	612 (86.5%)	6624

**Figure 5: Typical Detailed Statistics on Crypto Misuses.****Table 3: Statistics of disclosed misuses and responded vendors.**

Misuses	#	Vendors	#
Discovered misuses	6,624	All vendors	16
Disclosed misuses	1,835	Reported vendors	14
Confirmed misuses	299	Responded vendors	8
Assigned CVEs	2	Acknowledged our findings	4

portion of the misuses and the other 4 still evaluating them. Two CVEs have been assigned to the reported misuses. To streamline

the resolution process, we generate vendor-specific reports. Upon receiving consent from the vendors, these reports have been made accessible at [50].

We have received the following feedback: 1) Our approach had a low false positive rate of 1.3%, regarding the 299 confirmed misuses. We will discuss the details of these FPs in Section 5.4. 2) Some misuses were discovered in outdated firmware images, which are no longer supported by vendors. 3) Two vendors indicated that some misuses did exist but were difficult to prove exploitable. We would like to note that not all misuses are exploitable. The reasons for this

are twofold. Firstly, exploiting a misuse may depend on specific conditions. For example, performing chosen plaintext attacks may require multiple instances of plaintext and ciphertext, which is not always achievable in real-world scenarios. Secondly, triggering specific misuses, especially in hardware-dependent services, can prove challenging without direct physical access to the hardware.

5.3 (Q2) Case Study of Misuse Exploitation

To demonstrate the severity of the discovered misuses, we select two typical cases and conduct further manual analysis. These cases have been confirmed by the vendors and assigned CVEs. They are rated as critical or high severity by NVD (National Vulnerability Database). One of them has been fixed by the vendor, while the other is still in the process of being patched.

5.3.1 Case 1: Remote Device Control in All DrayTek Vigor Devices. DrayTek’s firmware uses a constant AES key, which comprises product model name and MAC address. Given the MAC address is enumerable, attackers can bind any device to their account, preventing the legitimate owner from registering it. Attackers can then generate WCF and DrayDDNS licenses and synchronize them from the official website. Our experiment found this zero-day vulnerability in 75 firmware images across 17 DrayTek devices.

We reported this vulnerability to DrayTek. They confirmed that all Vigor series products were affected by this vulnerability, spanning 64 devices and hundreds of firmware images. The NVD assigned this vulnerability CVE-2023-33778 and rated it as critical. As the time of writing this paper, DrayTek has addressed the issue and rewarded our findings.

5.3.2 Case 2: Privacy Leak and DNS Hijacking in NETGEAR Devices. A constant AES key is used in NETGEAR’s firmware, enabling attackers to decrypt intercepted data packets. This leaks sensitive information, including email, password, and device’s MAC and serial numbers. Further, attackers can exploit these details for DNS hijacking. Our investigation showed this vulnerability is present in an Apache HTTP server program, a core component deployed in 155 different firmware images.

We reported this vulnerability to NETGEAR, who confirmed it and acknowledged our findings. The NVD assigned it CVE-2023-34848*. NETGEAR rewarded our contribution through their bug bounty program (PSV-2020-0479 on Bugcrowd [5]). *The CVE is reserved but unpublished, pending NETGEAR’s resolution of the issue.

5.4 (Q3) Accuracy

The accuracy of our approach is evaluated from three aspects: manual inspection, vendor feedback, and the benchmark.

Manual Inspection. We arbitrarily select up to 20 samples per misuse type for manual inspection, totaling 276 samples. As we don’t hold the ground truth for real-world firmware misuses, we manually confirmed the misuses through code review using decompilation tools such as Ghidra and IDA Pro. As detailed in Table 4, we achieved a

Table 4: False positives on real-world firmware.

Misuses		V-01	V-02	V-03	V-04	V-05	V-06	V-07	V-08
Manual Inspection	Cases	20	9	20	20	20	20	20	20
	FPR	30.0%	11.1%	0.0%	0.0%	5.0%	25.0%	0.0%	0.0%
Vendor Feedback	Cases	2	-	57	125	-	6	3	-
	FPR	50.0%	-	0.0%	0.0%	-	50.0%	0.0%	-
Misuses		V-09	V-10	V-11	V-12	V-13	V-14	V-15	Total
Manual Inspection	Cases	20	9	18	20	20	20	20	276
	FPR	0.0%	0.0%	22.2%	0.0%	0.0%	0.0%	5.0%	6.5%
Vendor Feedback	Cases	5	-	-	16	44	10	31	299
	FPR	0.0%	-	-	0.0%	0.0%	0.0%	0.0%	1.3%

Table 5: Qualitative differences between CRYPTODY and CryptoREX. CT stands for CRYPTODY. CR stands for CryptoREX.

Feature	CR	CT	Feature	CR	CT
Cross-flow analysis	○	●	Types of crypto misuses	○	●
- cross-flow detection	○	●	- Symmetric ciphers	●	●
- considering contexts	○	●	- Asymmetric ciphers	○	●
Data-flow tracking	○	●	- Cryptographic hash	○	●
- Backward tracking	●	●	- PRNG	●	●
- Forward tracking	○	●	- HMAC	○	●
Extensibility	○	●	Cross-architecture analysis	○	●
- Crypto APIs	●	●	- ARM, MIPS	●	●
- Crypto policy rules	○	●	- x86, PowerPC, ...	○	●

precision rate of 93.5%. False positive rate varied based on misuse type. We divide the misuses into three categories: 1) Misuses that are detectable by matching function call sites exhibited no false positives, involving V-03, V-04, V-12, and V-14. 2) Misuses with an int type tracked variable, had a significantly low false positive rate, such as V-07, V-08, V-09, and V-10. Due to compiler optimization, the int type tracked variable is usually a immediate number, which is easy to track. 3) Misuses requiring sophisticated analysis techniques, such as involving multiple data flows, showed a higher false positive rate, like V-01, V-02, V-05, V-06, V-11, V-13, and V-15.

Vendor Feedback. We regard the feedback from IoT vendors, who possess definitive knowledge of their firmware, as the ground truth for our evaluation. Vendors have confirmed 299 reported misuses in total. Among them, only 4 were false positives, resulting in a low false positive rate of 1.3%. We list the details of these false positives below: 1) Three FPs were located in exception handler branches, which will not be executed under normal circumstances. Since our analysis does not factor in exception handling, these misuses are deemed as false positives. 2) One FP was caused by inaccurate decompilation by Ghidra, which is out of our scope.

Benchmark. For our developed benchmark, true positives are determined by comparing the results from our tool with the established ground truth. We conducted an evaluation of our system using CRYPTODYBENCH. Our tool successfully passed all test cases, exhibiting neither false positives nor false negatives.

5.5 (Q4) Comparison with State-of-the-Art

We present a qualitative and quantitative comparison between CRYPTODY and CryptoREX [55].

As illustrated in Table 5, CryptoREX cannot detect cross-flow misuses nor consider contextual information of APIs to improve accuracy due to the inherent limitations. For the data-flow tracking, CryptoREX supports only backward tracking, while CRYPTODY

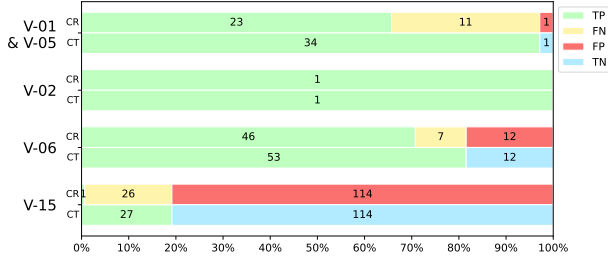


Figure 6: Quantitative differences between CRYPTODY and CryptoREX. CT stands for CRYPTODY. CR stands for CryptoREX.

supports both backward and forward tracking, which is essential for detecting cross-flow misuses. Regarding extensibility, our tool employs a decoupled design for enhanced extensibility, allowing users to easily add crypto policy rules using a declarative language rather than modifying the imperative code. For the supported types of crypto misuses, CRYPTODY offers broader coverage, capable of detecting more types of crypto misuses. While CryptoREX employs an IR to address cross-architecture analysis challenges, some of its operations, such as recovering stack layout and extracting dependency libraries from binaries, are still architecture-dependent. Hence, it does not achieve full cross-architecture analysis.

Figure 6 presents a quantitative comparison between CRYPTODY and CryptoREX (commit id 3dc81c9). To ensure a fair comparison, we adhered to two key criteria: 1) we evaluate both tools on the same data set maintained by [8], 2) the comparison is limited to crypto policy rules that are supported by CryptoREX. We select 162 binaries extracted from real-world firmware images, with their results validated by [8]. As shown in Figure 6, CRYPTODY outperforms CryptoREX in most rules, offering fewer false positives and detecting more crypto misuses. The reasons for this improved performance are twofold: 1) CRYPTODY exhibits enhanced data-flow tracking capabilities compared to CryptoREX. For example, CryptoREX does not correctly handle call-return edges of a CFG [8], leading to incorrect data-flow paths in interprocedural analysis. This deficiency results in a higher incidence of false positives, which are correctly flagged as true negatives by CRYPTODY. 2) CRYPTODY supports cross-flow analysis, which is essential for detecting intricate misuses. Specifically, for V-15, CRYPTODY is capable of identifying not only constant seeds but also weak and predictable seeds, such as those derived from the system time function.

5.6 (Q5) Efficiency

We assess the efficiency of CRYPTODY on both real-world firmware images and the CRYPTODYBENCH. The analysis of 1,431 firmware images took a total of 5 days and 6 hours. The average time for analyzing a single firmware image is 316 seconds, while a single ELF binary requires 18 seconds. For the CRYPTODYBENCH, the average execution times are 314 seconds on ARM, 297 seconds on MIPS, and 296 seconds on x86. Our statistics show that binary decompilation by Ghidra is the most time-intensive step in the process.

To improve efficiency, CRYPTODY employs a series of refinement strategies: 1) Our tool lifts the binary file to the IR on demand. We

mainly focus on binary files that directly or indirectly depend on crypto libraries. 2) We introduce a novel approach for cross-file analysis as outlined in Section 3.3.5. By reusing the summary of binary files and avoiding the construction of extensive cross-file graphs, we significantly reduce the time and memory consumption.

6 DISCUSSIONS

Limitations. Our approach has the following three limitations. Firstly, our solution focuses on Linux-based IoT firmware, which is a common type of available firmware in the market. Our approach can be generalized to other types of firmware, but it may require additional effort to adapt to the specific characteristics of different firmware. Secondly, we only consider standard crypto libraries widely employed in IoT firmware. As such, any misuse involving self-implemented crypto algorithms would not be detected by our approach. But we argue that it is a general best practice to utilize well-tested standard crypto libraries. Thirdly, our approach does not support the analysis of encrypted or obfuscated firmware images.

Threats to Validity. There are two main threats to the validity. Firstly, the API models are manually built by human experts based on the existing studies of crypto misuses in the literature and the documentation of crypto libraries. We have seen studies [56] that leverage natural language processing (NLP) techniques to automatically extract API models from the documentation of crypto libraries. However, we argue that such an approach is not suitable for our work for two reasons. 1) The documentation of crypto libraries is often incomplete and outdated, which is why developers tend to misuse crypto APIs [35]. In addition, crypto misuse studies are not documented in a consistent way, which makes it difficult to extract API models from them. 2) The generated API models are usually simple and difficult to generalize to support complex cases, such as cross-flow misuses. Hence, we believe that manually building API models is a more effective approach for our work. We have carefully examined the API models and the corresponding crypto libraries to ensure the correctness of the API models.

Secondly, because our approach relies on static binary analysis, it inevitably results in some false positives and false negatives. Regarding false positives, our tool might flag a misuse that is not executed at runtime, such as a non-executed branch. In addition, for V-11, the generated key might not be used at the same time. For false negatives, our tracking process has set a depth limit to accelerate analysis. This can lead us to miss misuses that are deeply nested in the code. However, we argue that the depth limit is sufficient to cover most misuses. Besides, our approach for identifying constant values is conservative. If a value changes but remains simple, such as a string value formed by concatenating two constant strings, we do not consider it a constant. As it is hard to statically determine the complexity of a value, we leave the analysis of such cases to future work.

Ethics and Responsible Disclosure. We have adhered closely to ethical guidelines throughout our study. Our data collection process involved gathering legitimate, publicly available firmware images from vendors' websites. We abstained from launching active attacks on any real-world IoT devices. Any zero-day vulnerabilities that we discovered were exploited solely in our isolated testbed, ensuring no external devices were affected.

As part of responsible disclosure, we reported 1,835 identified misuses to 14 vendors four months before submitting this paper. We have received responses from 8 vendors, with 4 confirming a portion of the misuses and the other 4 still evaluating them. Furthermore, we reported the discovered zero-day vulnerabilities to the corresponding vendors, who have acknowledged them and worked on fixes.

To aid vendors in addressing these issues, we have provided vendors with auto-generated reports, containing detailed information about the misuses and suggestions for fixes. With the vendor's consent, we have made some reports publicly accessible [50].

Ineffectual True Positives in Program Analysis. Ineffectual True Positives (ITPs) are considered true positives from the perspective of program analysis, yet their occurrence is permissible under the design specifications of particular protocols or within test code, suggesting no detrimental impact on the system. Identifying ITPs necessitates not only data-flow information but also supplementary inputs such as the design documentation of the specific protocol or expert knowledge concerning the protocol. Given that these issues are not widespread but rather confined to certain protocols, addressing them is deferred to future research endeavors. Nonetheless, the detection of ITPs retains value, as it holds potential to enhance the precision of analytical tools in subsequent developments.

7 RELATED WORK

Crypto Misuse Detection. Many research efforts have been devoted to detect crypto misuse bugs on various platforms [16, 28, 30, 31, 40, 42, 46, 48, 55]. Based on the analysis techniques used, existing solutions can be classified into two categories, static analysis [16, 28, 42, 46, 55] and dynamic analysis [30, 31, 40, 48]. Although dynamic analysis shows promising detection accuracy, it is not suitable for performing large-scale automatic analysis on IoT platforms for three reasons. *Firstly*, due to the lack of non-volatile memory parameters, firmware emulation has a low success rate [6]. *Secondly*, bare-metal analysis requires real devices and their debug ports. *Lastly*, considering that cryptography is often used for encryption and authentication in practice, many crypto functions usually require specific interaction, such as registering and login in, scanning QR codes, and connecting to WiFi or Bluetooth. Hence, it is challenging to trigger these functions automatically, leading to low crypto-related code coverage in real-world firmware. While static analysis solutions can achieve high code coverage, they, however, face the problem of false positives. The common thread of existing static analysis solutions is that they are designed for single data flow analysis, thus cannot handle cross-flow misuses.

Security Analysis of Firmware. In 2014, Costin et al. [14] performed the first large-scale analysis on firmware images to detect vulnerabilities without performing sophisticated static analysis. Later, Shoshitaishvili et al. [45] analyzed the authentication bypass vulnerabilities. Costin et al. [15] detected the network interfaces of embedded devices. KARONTE [43] concentrates on analyzing the insecure interactions among multiple binary files in firmware. Meanwhile, many dynamic analysis tools have been proposed to detect vulnerabilities in firmware, such as analyzing peripherals, drivers or firmware I/O [19, 47, 54], emulating and instrumenting embedded systems [6, 27, 32], testing and fuzzing [7, 49, 57].

Logic Inference. Logic inference has demonstrated its strength in security analysis, such as binary disassembly [21], memory forensics [41], C++ reverse engineering [44], fusing application and audit logs [53], and android analysis [24].

8 CONCLUSION

In this work, we abstract the intricate crypto misuse detection problem as a data-flow reasoning task. We propose CRYPTODY, a novel logic-inference-based framework for detecting crypto misuses via data-flow reasoning on multi-architecture IoT firmware images. CRYPTODY addresses the limitations of existing solutions by detecting cross-flow misuses and considering the contextual information of crypto APIs. It employs a series of strategies to improve the accuracy and the efficiency over the state-of-the-art. We evaluate our tool on large-scale real-world firmware images and our benchmark. We responsibly disclose the discovered misuses to the corresponding vendors and the findings have been acknowledged by the vendors. Two of the misuses have been assigned CVEs.

ACKNOWLEDGMENTS

We thank our shepherd and anonymous reviewers for their insightful comments. We thank Kaihang Ji, Zheng Leong Chua, and Chengyu Hu for their feedback on early drafts. We also thank Zhenbang Ma, Hang Yang, Cheng Liu, Yibing Qian, and Jiaqi Linghu for their help on our experiments. This research is supported by the National Research Foundation, Singapore, and Cyber Security Agency of Singapore under its National Cybersecurity R&D Programme (Fuzz Testing <NRF-NCR25-Fuzz-0001>), National Natural Science Foundation of China under Grant No. 62372268, Shandong Provincial Natural Science Foundation (No. ZR2021LZH007, No. ZR2022LZH013), and Jinan City "20 New Universities" Funding Project (2021GXRC084). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not reflect the views of the funding agencies.

REFERENCES

- [1] Yasemin Acar, Michael Backes, Sascha Fahl, Simson Garfinkel, Doowon Kim, Michelle L Mazurek, and Christian Stransky. 2017. Comparing the usability of cryptographic apis. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 154–171.
- [2] Sharmin Afrose, Sazzadur Rahaman, and Danfeng Yao. 2019. CryptoAPI-Bench: A Comprehensive Benchmark on Java Cryptographic API Misuses. In *2019 IEEE Cybersecurity Development (SecDev)*. IEEE, 49–61.
- [3] Amit Seal Ami, Nathan Cooper, Kaushal Kafle, Kevin Moran, Denys Poshyvanyk, and Adwait Nadkarni. 2022. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 614–631.
- [4] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *Acm Sigplan Notices* 49, 6 (2014), 259–269.
- [5] Bugcrowd. 2023. Bugcrowd: Crowdsourced Cybersecurity Platform. <https://www.bugcrowd.com/>
- [6] Daming D Chen, Maverick Woo, David Brumley, and Manuel Egele. 2016. Towards Automated Dynamic Analysis for Linux-based Embedded Firmware.. In *NDSS*, Vol. 16. 1–16.
- [7] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, Xiaofeng Wang, Wing Cheong Lau, Menghan Sun, Ronghai Yang, and Kehuan Zhang. 2018. IoTfuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing.. In *NDSS*.
- [8] Yikang Chen, Yibo Liu, Ka Lok Wu, Duc V Le, and Sze Yiu Chau. 2024. Towards Precise Reporting of Cryptographic Misuses. In *NDSS*.
- [9] MITRE Corporation. 2006. *Common Weakness Enumeration: A Community-Developed List of Software and Hardware Weakness Types*. <https://cwe.mitre.org/>

- [10] MITRE Corporation. 2006. *CWE-259: Use of Hard-coded Password*. <https://cwe.mitre.org/data/definitions/259.html>
- [11] MITRE Corporation. 2006. *CWE-337: Predictable Seed in Pseudo-Random Number Generator (PRNG)*. <https://cwe.mitre.org/data/definitions/337.html>
- [12] MITRE Corporation. 2006. *CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)*. <https://cwe.mitre.org/data/definitions/338.html>
- [13] MITRE Corporation. 2009. *CWE-780: Use of RSA Algorithm without OAEP*. <https://cwe.mitre.org/data/definitions/780.html>
- [14] Andrei Costin, Jonas Zaddach, Aurélien Francillon, and Davide Balzarotti. 2014. A large-scale analysis of the security of embedded firmwares. In *23rd {USENIX} Security Symposium ({USENIX} Security 14)*. 95–110.
- [15] Andrei Costin, Apostolis Zarras, and Aurélien Francillon. 2016. Automated dynamic firmware analysis at scale: a case study on embedded web interfaces. In *Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*. 437–448.
- [16] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. 2013. An empirical study of cryptographic misuse in android applications. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*. 73–84.
- [17] Stack Exchange. 2011. *Why should one not use the same asymmetric key for encryption as they do for signing?* <https://security.stackexchange.com/questions/1806/why-should-one-not-use-the-same-asymmetric-key-for-encryption-as-they-do-for-sig>
- [18] Stack Exchange. 2015. *Why is HMAC-SHA1 still considered secure?* <https://crypto.stackexchange.com/questions/26510/why-is-hmac-sha1-still-considered-secure>
- [19] Bo Feng, Alejandro Mera, and Long Lu. 2020. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In *Proceedings of the 29th USENIX Security Symposium*.
- [20] Felix Fischer, Konstantin Böttinger, Huang Xiao, Christian Stransky, Yasemin Acar, Michael Backes, and Sascha Fahl. 2017. Stack overflow considered harmful? the impact of copy&paste on android application security. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 121–136.
- [21] Antonio Flores-Montoya and Eric Schulte. 2020. Datalog disassembly. In *29th USENIX Security Symposium (USENIX Security 20)*. 1075–1092.
- [22] Ghidra. 2024. *Additional P-CODE Operations*. https://spinsel.dev/assets/2020-06-17-ghidra-brainfuck-processor-1/ghidra_docs/language_spec/html/additionalpcode.html
- [23] Damien Giry. 2020. *NIST Report on Cryptographic Key Length and Cryptoperiod (2020)*. <https://www.keylength.com/en/4/>
- [24] Grant Hernandez, Dave Jing Tian, Anurag Swarnim Yadav, Byron J Williams, and Kevin RB Butler. 2020. {BigMAC}::{Fine-Grained} Policy Analysis of Android Firmware. In *29th USENIX Security Symposium (USENIX Security 20)*. 271–287.
- [25] Tibor Jäger, Kenneth G Paterson, and Juraj Somorovsky. 2013. One Bad Apple: Backwards Compatibility Attacks on State-of-the-Art Cryptography. In *NDSS*.
- [26] Herbert Jordan, Bernhard Scholz, and Pavle Subotić. 2016. Soufflé: On synthesis of program analyzers. In *International Conference on Computer Aided Verification*. Springer, 422–430.
- [27] Karl Koscher, Tadayoshi Kohno, and David Molnar. 2015. SURROGATES: Enabling near-real-time dynamic analyses of embedded systems. In *9th {USENIX} Workshop on Offensive Technologies ({WOOT} 15)*.
- [28] Stefan Krüger, Johannes Späth, Karim Ali, Eric Bodden, and Mira Mezini. 2018. CrysL: An extensible approach to validating the correct usage of cryptographic apis. In *32nd European Conference on Object-Oriented Programming (ECOOP 2018)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- [29] ReFirm Labs. 2014. *Binwalk: Firmware Analysis Tool*. <https://github.com/ReFirmLabs/binwalk>
- [30] Juanru Li, Zhiqiang Lin, Juan Caballero, Yuanyuan Zhang, and Dawu Gu. 2018. K-Hunt: Pinpointing insecure cryptographic keys from execution traces. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 412–425.
- [31] Yong Li, Yuanyuan Zhang, Juanru Li, and Dawu Gu. 2015. iCryptoTracer: Dynamic analysis on misuse of cryptography functions in iOS applications. In *International Conference on Network and System Security*. Springer, 349–362.
- [32] Qiang Liu, Cen Zhang, Lin Ma, Muhui Jiang, Yajin Zhou, Lei Wu, Wenbo Shen, Xiapu Luo, Yang Liu, and Kui Ren. 2021. Firmguide: Boosting the capability of rehosting embedded linux kernels through model-guided kernel execution. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 792–804.
- [33] Na Meng, Stefan Nagy, Danfeng Yao, Wenjie Zhuang, and Gustavo Arango Argoty. 2018. Secure coding practices in java: Challenges and vulnerabilities. In *Proceedings of the 40th International Conference on Software Engineering*. 372–383.
- [34] Joydeep Mitra and Venkatesh-Prasad Ranganath. 2017. Ghera: A repository of android app vulnerability benchmarks. In *Proceedings of the 13th International Conference on Predictive Models and Data Analytics in Software Engineering*. 43–52.
- [35] Sarah Nadi, Stefan Krüger, Mira Mezini, and Eric Bodden. 2016. Jumping through hoops: Why do Java developers struggle with cryptography APIs?. In *Proceedings of the 38th International Conference on Software Engineering*. 935–946.
- [36] National Security Agency (NSA). 2019. *Ghidra: A Software Reverse Engineering (SRE) Suite of Tools Developed by NSA*. <https://ghidra-sre.org/>
- [37] National Security Agency (NSA). 2019. *P-Code Reference Manual*. <https://ghidra.re/courses/languages/html/pcoderef.html>
- [38] National Institute of Standards and Technology (NIST). 2017. *Juliet Test Suite: A collection of test cases in the C/C++ language*. <https://samate.nist.gov/SARD/testsuite.php>
- [39] Stack Overflow. 2011. *How to generate RSA private key using OpenSSL?* <https://stackoverflow.com/questions/5927164/how-to-generate-rsa-private-key-using-openssl>
- [40] Luca Piccolboni, Giuseppe Di Guglielmo, Luca P. Carloni, and Simha Sethumadhavan. 2021. CRYLOGGER: Detecting Crypto Misuses Dynamically. In *2021 IEEE Symposium on Security and Privacy (SP)*. 1972–1989. <https://doi.org/10.1109/SP40001.2021.00010>
- [41] Zhenxiao Qi, Yu Qu, and Heng Yin. 2022. LogicMEM: Automatic profile generation for binary-only memory forensics via logic inference. In *Proceedings 2022 Network and Distributed System Security Symposium*. Internet Society.
- [42] Sazzadur Rahaman, Ya Xiao, Sharmin Afrose, Fahad Shaon, Ke Tian, Miles Frantz, Murat Kantarcioglu, and Danfeng Yao. 2019. Cryptoguard: High precision detection of cryptographic vulnerabilities in massive-sized Java projects. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2455–2472.
- [43] Nilo Redini, Aravind Machiry, Ruoyu Wang, Chad Spensky, Andrea Continella, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2020. Karonte: Detecting insecure multi-binary interactions in embedded firmware. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1544–1561.
- [44] Edward J Schwartz, Cory F Cohen, Michael Duggan, Jeffrey Gennari, Jeffrey S Havrilla, and Charles Hines. 2018. Using logic programming to recover c++ classes and methods from compiled executables. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. 426–441.
- [45] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, and Giovanni Vigna. 2015. Firmalice-automatic detection of authentication bypass vulnerabilities in binary firmware. In *NDSS*.
- [46] Shao Shuai, Dong Guowei, Guo Tao, Yang Tianchang, and Shi Chenjie. 2014. Modelling analysis and auto-detection of cryptographic misuse in android applications. In *2014 IEEE 12th International Conference on Dependable, Autonomic and Secure Computing*. IEEE, 75–80.
- [47] Dokyung Song, Felicitas Hetzelt, Dipanjan Das, Chad Spensky, Yeoul Na, Stijn Volckaert, Giovanni Vigna, Christopher Kruegel, Jean-Pierre Seifert, and Michael Franz. 2019. Periscope: An effective probing and fuzzing framework for the hardware-os boundary. In *NDSS*.
- [48] David Sounthiraraj, Justin Sahs, Garret Greenwood, Zhiqiang Lin, and Latifur Khan. 2014. Smv-hunter: Large scale, automated detection of ssl/tls man-in-the-middle vulnerabilities in android apps. In *In Proceedings of the 21st Annual Network and Distributed System Security Symposium (NDSS'14)*.
- [49] Prashast Srivastava, Hui Peng, Jiahao Li, Hamed Okhravi, Howard Shrobe, and Mathias Payer. 2019. FirmFuzz: automated IoT firmware introspection and analysis. In *Proceedings of the 2nd International ACM Workshop on Security and Privacy for the Internet-of-Things*. 15–21.
- [50] CrypTody. 2024. *Artifacts for CrypTody*. <https://github.com/J4n1ng/CrypTody>
- [51] Wikipedia. 2023. *Converting to SSA*. https://en.wikipedia.org/wiki/Static_single_assignment_form#Converting_to_SSA
- [52] Wikipedia. 2023. *Coppersmith's attack*. https://en.wikipedia.org/wiki/Coppersmith%27s_attack
- [53] Le Yu, Shiqing Ma, Zhuo Zhang, Guanhong Tao, Xiangyu Zhang, Dongyan Xu, Vincent E Urias, Han Wei Lin, Gabriela F Ciocarlie, Vinod Yegneswaran, et al. 2021. ALchemist: Fusing Application and Audit Logs for Precise Attack Provenance without Instrumentation. In *NDSS*.
- [54] Jonas Zaddach, Luca Bruno, Aurelien Francillon, Davide Balzarotti, et al. 2014. AVATAR: A Framework to Support Dynamic Security Analysis of Embedded Systems' Firmwares. In *NDSS*, Vol. 14. 1–16.
- [55] Li Zhang, Jiongqi Chen, Wenrui Diao, Shanqing Guo, Jian Weng, and Kehuan Zhang. 2019. CryptoREX: Large-scale Analysis of Cryptographic Misuse in IoT Devices. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*. 151–164.
- [56] Binbin Zhao, Shouling Ji, Xuhong Zhang, Yuan Tian, Qinying Wang, Yuwen Pu, Chenyang Lyu, and Raheem Beyah. 2023. UVSCAN: Detecting Third-Party Component Usage Violations in IoT Firmware. In *32nd USENIX Security Symposium (USENIX Security 23)*. 3421–3438.
- [57] Yaowen Zheng, Ali Davanian, Heng Yin, Chengyu Song, Hongsong Zhu, and Limin Sun. 2019. FIRM-AFL: high-throughput greybox fuzzing of iot firmware via augmented process emulation. In *28th USENIX Security Symposium (USENIX Security 19)*. 1099–1114.

A API MODEL

We manually crafted 329 API models covering 7 common crypto libraries on IoT platforms, including libcrypto (OpenSSL), libgcrypt, wolfcrypt, libmccrypt, cryptlib, libcrypt, and libc. Each API model contains a function signature, a parameter of interest, the corresponding library, the misuse type, and constraints on the parameter if applicable. For instance, the API model of `RSA_generate_key_ex` in libcrypto is shown below. The model specifies that the second parameter, namely `bits`, should not be less than 2048, which is a common misuse of using keys of insufficient size (V-08).

```

1  {
2      "func_name": "RSA_generate_key_ex",
3      "parameters": {
4          "1": "RSA *rsa",
5          "2": "int bits",
6          "3": "BIGNUM *e",
7          "4": "BN_GENCB *cb"
8      },
9      "return_type": "int",
10     "rule_id": "a_key_len",
11     "parameter_index": 2,
12     "parameter_type": "int",
13     "parameter_value": "<2048",
14     "library": "libcrypto",
15     "supplement": ""
16 }
```

API models formatted in JSON are encoded into Datalog facts to facilitate easy extension and modification, thus enabling users to effortlessly add new models or modify existing ones in response to the evolving crypto APIs. The above API model will be encoded as facts in the Datalog format as shown below (line 2).

```

1  EVP_PKEY_CTX_set_rsa_keygen_bits 2
2  RSA_generate_key_ex 2
3  RSA_generate_key 1
```

B IOT-SPECIFIC BENCHMARK

To assess the accuracy of the CRYPTODY tool, we create an IoT-specific crypto misuse detection benchmark, termed CRYPTODY-BENCH. This benchmark includes 65 unit test cases, with 46 positive cases featuring crypto misuses and 19 negative cases without such issues, as shown in Table 6. We outline the key features as follows:

- It covers 12 types of crypto misuses, such as symmetric and asymmetric ciphers, hash functions, and PRNG, mainly utilizing APIs from OpenSSL.
- It provides 23 basic intraprocedural cases and 42 complex cases, including interprocedural and cross-file cases.
- It contains source code and multi-architecture binary code in the formats of x86, ARM, and MIPS, making it suitable for cross-architecture analysis tools designed for IoT devices.

Table 6: Overview of CRYPTODYBENCH. Remarks: Intra. stands for intraprocedural. Inter. stands for interprocedural. CF stands for cross-file.

Misuses	Type	Tracking Method			Ground Truth		Total
		Intra.	Inter.	CF	Positive	Negative	
V-01 Use constant cryptographic keys	Symmetric Ciphers	3	4	2	6	3	9
V-02 Use static IVs in CBC mode		3	4	2	6	3	9
V-03 Use ECB mode		1	0	1	2	0	2
V-04 Use insecure symmetric ciphers		1	0	1	2	0	2
V-05 Use constant passwords for PBE		2	4	0	4	2	6
V-06 Use constant salts for PBE		2	4	0	4	2	6
V-07 Fewer than 1,000 iterations for PBE		2	4	0	4	2	6
V-08 Use keys of insufficient size	Asymmetric Ciphers	2	4	2	6	2	8
V-09 Use RSA algorithm without OAEP		2	4	0	4	2	6
V-10 Use low public exponents in RSA		2	4	0	4	2	6
V-12 Use insecure hash algorithms	Hash	1	0	1	2	0	2
V-15 Use predictable or constant PRNG seeds	PRNG	2	1	0	2	1	3
Total		23	33	9	46	19	65